PYTHON BACKEND ENGINEERING

Data Structures & Decisions

Class 02 · Complete Student Notes

LEVEL
Beginner → Curious

TODAY
List · Tuple · Set · Dict · if

INSIDE
Concepts · Examples · Practice

Where we are. Last class covered variables, types, input, and errors. Today your programs learn to *hold collections of data* — lists, tuples, sets, and dictionaries — and to *make decisions* with `if / elif / else`. These four data structures are the backbone of every backend you'll ever build. Type every example, and finish the green practice boxes.

1 Quick Recap — Variables & Types

A quick warm-up from Class 1. A variable stores one value, and every value has a type:

PYTHON

```python
name = "Saroar"   # str
age = 29          # int
salary = 90000.50  # float
is_active = True   # bool
```

Quick check — is "29" the same as 29?

PYTHON

```python
print("29" == 29)   # False
```

Text and numbers are different types, so `"29"` (a string) is never equal to `29` (a number).

2 Lists — Creating & Indexing

A **list** is a collection that holds many values together, in order, inside square brackets:

PYTHON

```python
numbers = [10, 20, 30, 40, 50]
print(numbers)
print(type(numbers))   # <class 'list'>
```

Each item has a position — an **index**. Python counts from `0`. It also counts backwards with negative indexes:




```python
print(numbers[0])    # 10  (first)
print(numbers[2])    # 30
print(numbers[-1])   # 50  (last)
print(numbers[-2])   # 40
```

3 Lists Are Mutable

"Mutable" means you can change a list after creating it — including replacing an item by its index:

```python
numbers = [10, 20, 30]
numbers[0] = 100
print(numbers)   # [100, 20, 30]
```

> 🔬 **Go Deeper — mutability has a catch**
>
> Because a variable is a *label* pointing at the list (Class 1), writing `b = a` gives the **same** list two names — change one and the other changes too. Use `b = a.copy()` for an independent copy. Numbers and strings are immutable, so they never surprise you this way.

4 Lists Can Hold Anything

A list can mix types, and can even contain other lists (a **nested list**):

PYTHON

```python
data = [10, "Python", 3.14, True]

students = [
    ["Rahim", 20],
    ["Karim", 22],
    ["Hasan", 21],
]
print(students[0])       # ['Rahim', 20]
print(students[0][0])    # Rahim
```

Reading nested access

`students[0][0]` means: take item 0 of `students` (the first pair), then item 0 of *that* (the name).

5 List Slicing

A **slice** grabs a range of items: `list[start:end:step]`. The `end` index is *not* included.

```python
numbers = [10, 20, 30, 40, 50]

print(numbers[1:4])    # [20, 30, 40]
print(numbers[:3])     # [10, 20, 30]  (from start)
print(numbers[2:])     # [30, 40, 50]  (to end)
print(numbers[::2])    # [10, 30, 50]  (every 2nd)
print(numbers[::-1])   # [50, 40, 30, 20, 10]  (reversed!)
```

🔬 **Go Deeper — the reverse trick**

`numbers[::-1]` is the shortest way to reverse a list (or a string!). The negative step walks through the items backwards.

PRACTICE 5

1. From `[10,20,30,40,50]`, slice out `[20,30]`.
2. Print the same list reversed using a slice.

6 List Methods

Methods are actions you call on a list with a dot:

Method	What it does
`.append(x)`	Add `x` to the end
`.insert(i, x)`	Insert `x` at index `i`
`.extend([...])`	Add each item of another list
`.remove(x)`	Remove the first matching `x`
`.pop(i)`	Remove & return item at `i` (last if empty)
`.sort()` / `.reverse()`	Sort / reverse in place
`.count(x)` / `.index(x)`	Count occurrences / find position

PYTHON

```python
numbers = [50, 10, 30, 20]
numbers.append(40)          # [50,10,30,20,40]
numbers.sort()              # [10,20,30,40,50]
numbers.sort(reverse=True)  # [50,40,30,20,10]
last = numbers.pop()        # removes & returns 10
```

append vs extend — a classic mix-up

PYTHON

```python
a = [1, 2, 3]
a.append([4, 5])   # [1, 2, 3, [4, 5]]  ← adds ONE item (a list)

b = [1, 2, 3]
b.extend([4, 5])   # [1, 2, 3, 4, 5]  ← adds each item
```

Use `append` to add one thing; `extend` to merge in many.

7 Membership: in / not in

Check whether a value exists in a collection — it returns `True` / `False`:

PYTHON

```python
names = ["Rahim", "Karim", "Hasan"]
print("Rahim" in names)       # True
print("Sakib" not in names)   # True
```

You'll use this constantly

`in` is the natural way to ask “does this exist?” before you act on it — you'll pair it with `if` shortly.

8 Tuples — Like Lists, but Locked

A **tuple** looks like a list but uses parentheses, and it's **immutable** — once created, it can't be changed.

```python
point = (10, 20, 30, 40)
print(type(point))   # <class 'tuple'>
print(point[0])      # 10
print(point[1:3])    # (20, 30)
```

Tuples can't be modified

```python
point = (10, 20, 30)
point[0] = 100

# TypeError: 'tuple' object does not support item assignment
```

A list would allow `list[0] = 100` — a tuple refuses. That “lock” is the whole point.

9 Tuple Unpacking

You can pull a tuple's values into separate variables in one line — clean and very common:

```python
student = ("Rahim", 25, "CSE")
name, age, dept = student
print(name)   # Rahim
print(dept)   # CSE

x, y = (10, 20)   # x=10, y=20
```

10 List vs Tuple

List	Tuple
Written with `[ ]`	Written with `( )`
Mutable — can change	**Immutable** — locked
For data that changes	For fixed data
`["Rahim", "Karim"]`	`(23.8, 90.4)`

When to pick which

Use a **list** when items get added or removed (a cart, a list of users). Use a **tuple** for a fixed pair or record that should never change — like GPS `(latitude, longitude)`.

11 Sets — Unique Values Only

A **set** is a collection of *unique* values in curly braces. Duplicates are automatically dropped:

```python
nums = {1, 2, 2, 3, 3, 3}
print(nums)   # {1, 2, 3}
```

That makes sets perfect for removing duplicates from a list:

```python
numbers = [1, 2, 2, 3, 4, 4, 5]
unique = list(set(numbers))
print(unique)   # [1, 2, 3, 4, 5]
```

12 Set Operations

Sets support real mathematical set operations — great for comparing two groups:

```python
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a | b)   # {1,2,3,4,5,6}  union (everything)
print(a & b)   # {3, 4}          intersection (common)
print(a - b)   # {1, 2}          difference (in a, not b)
print(a ^ b)   # {1,2,5,6}       symmetric (not shared)
```

PRACTICE 12

1. Given two sets of student names, find the ones present in **both** (intersection).

13 Dictionaries — Key → Value Data

A **dictionary** stores data as **key–value pairs**. Instead of a numeric index, you look items up by a meaningful key:

```python
student = {
    "name": "Rahim",
    "age": 22,
    "department": "CSE",
    "cgpa": 3.5,
}
print(student["name"])   # Rahim
```

Dictionaries are mutable — change a value, or add a brand-new key:

```python
student["age"] = 23              # change existing
student["university"] = "GUB"    # add new key
```

14 Dictionary Methods

Method	Returns
`.get("key")`	The value, or `None` if the key is missing (safe)
`.keys()`	All the keys
`.values()`	All the values
`.items()`	All key–value pairs
`.pop("key")`	Removes that key and returns its value

.get() vs [ ] — avoid the KeyError

```python
print(student.get("phone"))   # None  (safe, no crash)
print(student["phone"])       # KeyError: 'phone'  (crashes!)
```

Use `.get()` when a key *might* be missing; use `[ ]` when you're sure it exists.

15 Nested Data — the shape of real APIs

Dictionaries can hold dictionaries, and lists can hold dictionaries. This is exactly how real backend data looks.

PYTHON — NESTED DICT

```python
user = {
    "id": 101,
    "name": "Rahim",
    "address": {"city": "Dhaka", "country": "Bangladesh"},
}
print(user["address"]["city"])   # Dhaka
```

PYTHON — LIST OF DICTS

```python
users = [
    {"id": 1, "name": "Rahim", "age": 25},
    {"id": 2, "name": "Karim", "age": 28},
]
print(users[0]["name"])   # Rahim
print(users[1]["age"])    # 28
```

🔬 **Go Deeper — this IS JSON**

A “list of dictionaries” is almost exactly the JSON that web APIs send and receive. When you build APIs later, you'll be shaping Python lists and dicts and handing them out as JSON. Learning this structure now pays off directly.

16 Making Decisions: if / elif / else

An `if` runs a block only when its condition is `True`. Add `elif` for more paths and `else` for "none of the above". Python runs the **first** true branch and skips the rest.

PYTHON

```python
marks = 75

if marks >= 80:
    print("A+")
elif marks >= 70:
    print("A")
elif marks >= 60:
    print("A-")
else:
    print("Fail")
```

Why does 75 print "A"?

The first condition (`>= 80`) is False, the second (`>= 70`) is True — so Python runs that branch and stops. This is why the **strictest condition must come first**.

17 Comparison & Logical Operators

Comparison

`==` `!=` `>` `<` `>=` `<=`

Logical

`and` `or` `not`

```python
age = 25
salary = 50000

if age >= 18 and salary >= 30000:
    print("Eligible")     # and → both must be True

is_blocked = False
if not is_blocked:
    print("User can login")   # not → flips the value
```

Remember

`and` = both true. `or` = at least one true. `not` = flip True/False.

Real-world login check:

PYTHON — LOGIN.PY

```python
username = "admin"
password = "1234"

if username == "admin" and password == "1234":
    print("Login successful")
else:
    print("Invalid username or password")
```

18 Nested If & Truthy / Falsy

You can put an `if` inside another, but often a combined condition reads better:

```python
# nested
if age >= 18:
    if has_ticket:
        print("You can enter")

# cleaner — same result
if age >= 18 and has_ticket:
    print("You can enter")
```

Any value can be tested for truth. Empty things are “falsy”:

```python
bool(0)      # False
bool("")     # False
bool([])     # False
bool({})     # False
bool(None)   # False

name = ""
if name:                    # empty string is falsy
    print("Name exists")
else:
    print("Name is empty")
```

Pythonic style

Prefer `if not numbers:` over `if numbers == []:` to check for an empty list — shorter and clearer.

19 Conditions + Data Structures

Now combine everything. This is what real backend logic looks like.

PYTHON — WITH A LIST

```python
numbers = [10, 20, 30, 40]
if 30 in numbers:
    print("30 exists")
```

PYTHON — WITH A DICT

```python
user = {"name": "Rahim", "age": 25}
if user.get("email"):
    print(user["email"])
else:
    print("Email not found")
```

PYTHON — WITH A SET (AUTHORIZATION)

```python
allowed_roles = {"admin", "manager", "hr"}
role = "admin"
if role in allowed_roles:
    print("Access granted")
else:
    print("Access denied")
```

> 🔬 **Go Deeper — this is authorization**
>
> Checking `if role in allowed_roles` is exactly how backends decide who's allowed to do what. Sets make this check fast even with thousands of roles, because looking up membership in a set is near-instant.

20 Common Mistakes

Wrong ✗	Right ✓
`if age = 18:`	`if age == 18:`
`if x:` then unindented body	Indent the body 4 spaces
`if numbers == []:`	`if not numbers:`
`student["phone"]` (may crash)	`student.get("phone")`

The #1 beginner bug

`=` assigns a value; `==` compares. Using `=` inside an `if` is a syntax error in Python — a helpful safety net.

21 Concept Map

The four data structures at a glance:

📋 List

- Ordered
- Mutable
- Duplicates allowed
- [1, 2, 3]

🔒 Tuple

- Ordered
- Immutable
- Duplicates allowed
- (1, 2, 3)

✨ Set

- Unordered
- Mutable
- Unique only
- {1, 2, 3}

🗂️ Dict

- Key → Value
- Mutable
- Unique keys
- {"a": 1}

Decision making

`if` → `if-else` → `if-elif-else` → nested `if`, all powered by comparison (`== != > <`) and logical (`and / or / not`) operators.

22 Putting It Together — Shopping Cart

Lists + dictionaries + conditions working as one. (The `for` loop is a sneak peek at next class.)

PYTHON — CART.PY

```python
cart = [
    {"name": "Keyboard", "price": 2000, "quantity": 2},
    {"name": "Mouse", "price": 1000, "quantity": 1},
]

total = 0
for item in cart:
    total += item["price"] * item["quantity"]

if total >= 5000:
    discount = total * 0.10
else:
    discount = 0

print("Total:", total)
print("Final:", total - discount)
```

🎯 IN-CLASS ASSIGNMENT

Student Grading System

Write a program called `grading.py` that:

- Asks the user for their **name** and their **marks**.
- First checks the marks are valid (0–100); if not, prints `"Invalid marks"`.
- Otherwise prints the grade using `if / elif / else`.

GRADING SCALE

80+ → A+ 70–79 → A 60–69 → A- 50–59 → B 40–49 → C <40 → F

EXPECTED OUTPUT

```
Saroar, your grade is A
```

Before you submit

Convert marks with `int()`. Check the invalid range **first**, and order the grade branches from highest to lowest. Test with 95, 72, 45, and 150.

🔬 **Stretch goal**

Use an f-string for the message: `print(f"{name}, your grade is {grade}")`. It slots variables straight into the text.

📚 TAKE-HOME PRACTICE

Ten Problems to Submit

Do these **at home** and submit them together. Put each solution in its own `.py` file inside a GitHub repo named `python-homework-2`, then share the link. Everything you need is in these notes.

Beginner

1 First & last element

From `[10, 20, 30, 40, 50]`, print the first and last elements (use indexing, including `-1`).

2 Does it exist?

Ask the user for a number, then check with `in` whether it's in a given list. Print a friendly yes/no message with `if/else`.

3 Tuple max & min

Given a tuple of numbers, print its maximum and minimum (hint: `max()` and `min()` work on tuples too).

4 Remove duplicates

From `[1, 2, 2, 3, 3, 4, 5, 5]`, produce a list of unique values using `set()`.

5 Read a dictionary

Make a student dictionary, then print the name and age by their keys.

Intermediate

6 Grade from marks

Extend the in-class grading system: validate 0–100, then print the grade with `if/elif/else`.

7 Voting eligibility

Given an `age` and a boolean `is_citizen`, print whether the person can vote — must be 18+ **and** a citizen (use `and`).

8 Login system

Store a username and password, ask the user to enter them, and print success or failure using `and`.

9 Safe key check

Check whether a dictionary has an `"email"` key. Print it if present, otherwise `"Email not found"` — use `.get()` or `in`.

10 Common elements

Given two sets, print the values that appear in **both** (intersection with `&`).

> **Submission checklist**
>
> Ten `.py` files · one repo (`python-homework-2`) · each committed with a clear message · pushed and visible on GitHub · link submitted.

Coming up next class → loops (`for` and `while`) to process these collections automatically, and functions to package your code.